EXPERIMENT 6

D15A-57

Aim: Apply K-Means and Hierarchical Clustering on sample datasets

Theory:

1. Dataset Source
Custom dataset : Link

2. Dataset Description

The dataset contains time-series fitness activity records collected from wearable fitness trackers.

Features used in this experiment:

1. timestamp – Date and time at which the activity was recorded
2. steps – Number of steps recorded
3. calories – Calories burned
4. data source – Source of data collection (not used in clustering)

After preprocessing, the data was aggregated on a daily basis:

- Two numerical features:
  - Daily total steps
  - Daily total calories burned

3. Mathematical Formulation of the Algorithm

A) K-Means Clustering

Objective:
Minimize the within-cluster sum of squares (WCSS).

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

- k = number of clusters
- Ci = set of points in cluster iii
- μi= centroid (mean) of cluster iii

Algorithm steps:

1. Initialize k centroids randomly.
2. Assign each data point to the nearest centroid.
3. Recompute centroids as the mean of assigned points.
4. Repeat steps 2 and 3 until convergence.

B) Hierarchical Clustering

Distance Metric (Euclidean Distance):

$$d(x, y) = \sqrt{\sum_{j=1}^{d} (x_j - y_j)^2}$$

Ward linkage merges clusters such that the increase in total within-cluster variance is minimized.

Ward's criterion:

$$\Delta E = \frac{n_a n_b}{n_a + n_b} \|\mu_a - \mu_b\|^2$$

Where:

- na, nb = number of points in clusters a and b
- μa, μb = centroids of clusters

Clusters are formed by successively merging the closest clusters until all points form a single cluster. The dendrogram helps determine the number of clusters.

4. Algorithm Limitations

A) K-Means Limitations

- Requires predefined number of clusters (k).
- Sensitive to outliers.
- Assumes spherical clusters.
- Performs poorly for non-linear cluster boundaries.
- Results depend on initial centroid placement.

B) Hierarchical Clustering Limitations

- Computationally expensive for large datasets.
- Once clusters are merged, they cannot be split.
- Sensitive to noise and outliers.
- Memory intensive.

5. Methodology / Workflow

Step 1: Load multiple CSV files containing steps and calories data.

Step 2: Merge the datasets.

Step 3: Remove unnecessary columns (such as data source).

Step 4: Convert timestamps into date format.

Step 5: Aggregate steps and calories to obtain daily totals.

Step 6: Standardize features using feature scaling.

Step 7: Apply K-Means clustering with k = 3.

Step 8: Apply Hierarchical clustering using Ward linkage.

Step 9: Interpret clusters based on average daily steps:

- Low Activity

- Moderate Activity

- High Activity

Step 10: Visualize clusters using scatter plots and dendrogram.

6. Performance Analysis

Since clustering is an unsupervised learning technique, traditional metrics such as accuracy and precision were not used.

In this experiment:

- K-Means clustering was applied with k = 3.
- Cluster quality was evaluated using the Silhouette Score.
- The Silhouette Score was computed on the standardized feature set.
- The score was analyzed to determine how well-separated the clusters were.

A higher Silhouette Score indicates better-defined clusters.

7. Hyperparameter Tuning

A) K-Means

For K-Means, the number of clusters (k) was tuned using the Elbow Method and by comparing Silhouette Scores for different values of k.

Based on the analysis, k = 3 was selected as it provided the best cluster separation and interpretability.

Multiple initializations were used to ensure stable and consistent clustering results.

B) Hierarchical Clustering

For Hierarchical Clustering, different linkage methods were considered, and Ward linkage was selected.

The number of clusters was determined using the dendrogram.

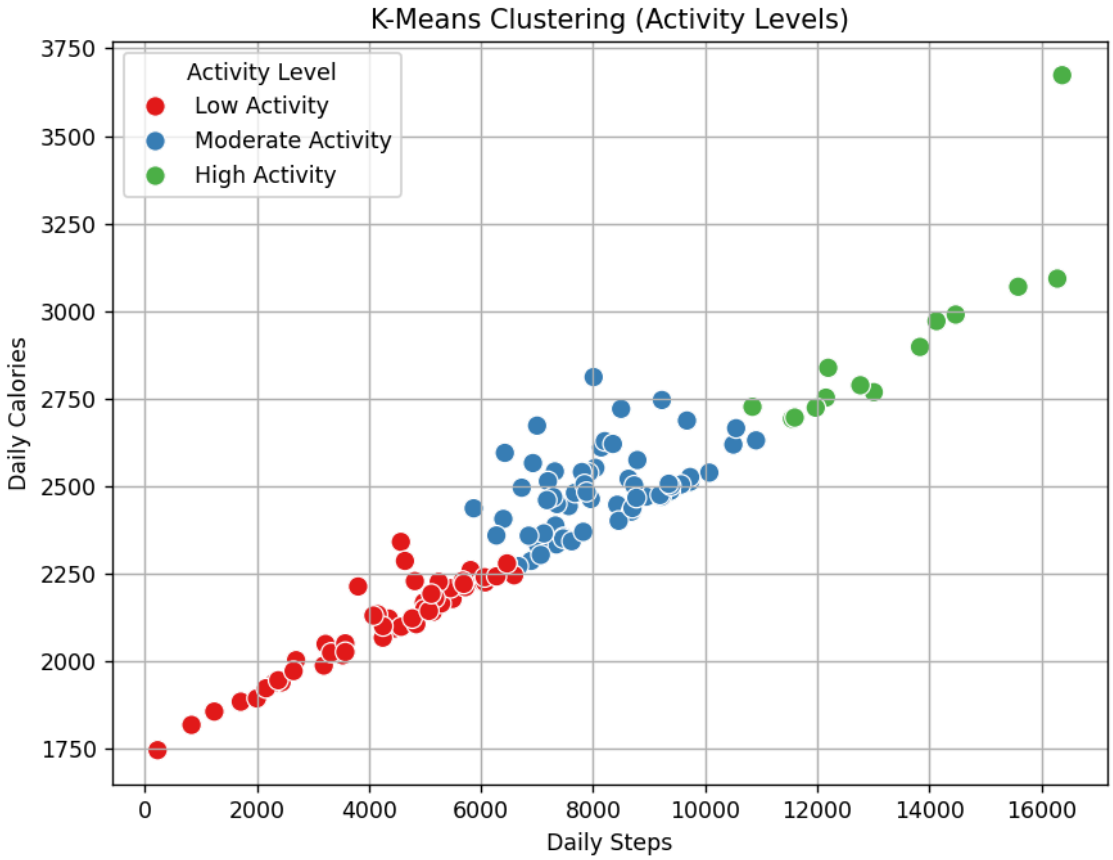
Ward linkage was chosen because it produced compact and well-separated clusters, making the results more interpretable.

Code: K-means

```python
import pandas as pd
import glob
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

step_files = glob.glob("steps_data/*.csv")
cal_files = glob.glob("calories_data/*.csv")

steps_df = pd.concat([pd.read_csv(f) for f in step_files], ignore_index=True)
cal_df = pd.concat([pd.read_csv(f) for f in cal_files], ignore_index=True)

steps_df = steps_df[['timestamp', 'steps']]
cal_df = cal_df[['timestamp', 'calories']]

steps_df['timestamp'] = pd.to_datetime(steps_df['timestamp'])
cal_df['timestamp'] = pd.to_datetime(cal_df['timestamp'])


steps_df['date'] = steps_df['timestamp'].dt.date
cal_df['date'] = cal_df['timestamp'].dt.date

daily_steps = steps_df.groupby('date')['steps'].sum().reset_index()
daily_cal = cal_df.groupby('date')['calories'].sum().reset_index()

daily = pd.merge(daily_steps, daily_cal, on='date')

print("Total Days:", len(daily))


X = daily[['steps', 'calories']]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```python
kmeans = KMeans(n_clusters=3, random_state=42)
daily['KMeans_Cluster'] = kmeans.fit_predict(X_scaled)

k_means_avg = daily.groupby('KMeans_Cluster')['steps'].mean()
sorted_k = k_means_avg.sort_values().index.tolist()

k_labels = {
    sorted_k[0]: "Low Activity",
    sorted_k[1]: "Moderate Activity",
    sorted_k[2]: "High Activity"
}

daily['KMeans_Label'] = daily['KMeans_Cluster'].map(k_labels)


sil_score = silhouette_score(X_scaled, daily['KMeans_Cluster'])

print("Silhouette Score:", sil_score)

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=daily,
    x='steps',
    y='calories',
    hue='KMeans_Label',
    palette='Set1',
    s=80
)

plt.title("K-Means Clustering (Activity Levels)")
plt.xlabel("Daily Steps")
plt.ylabel("Daily Calories")
plt.legend(title="Activity Level")
plt.grid(True)
plt.show()
```

Output:



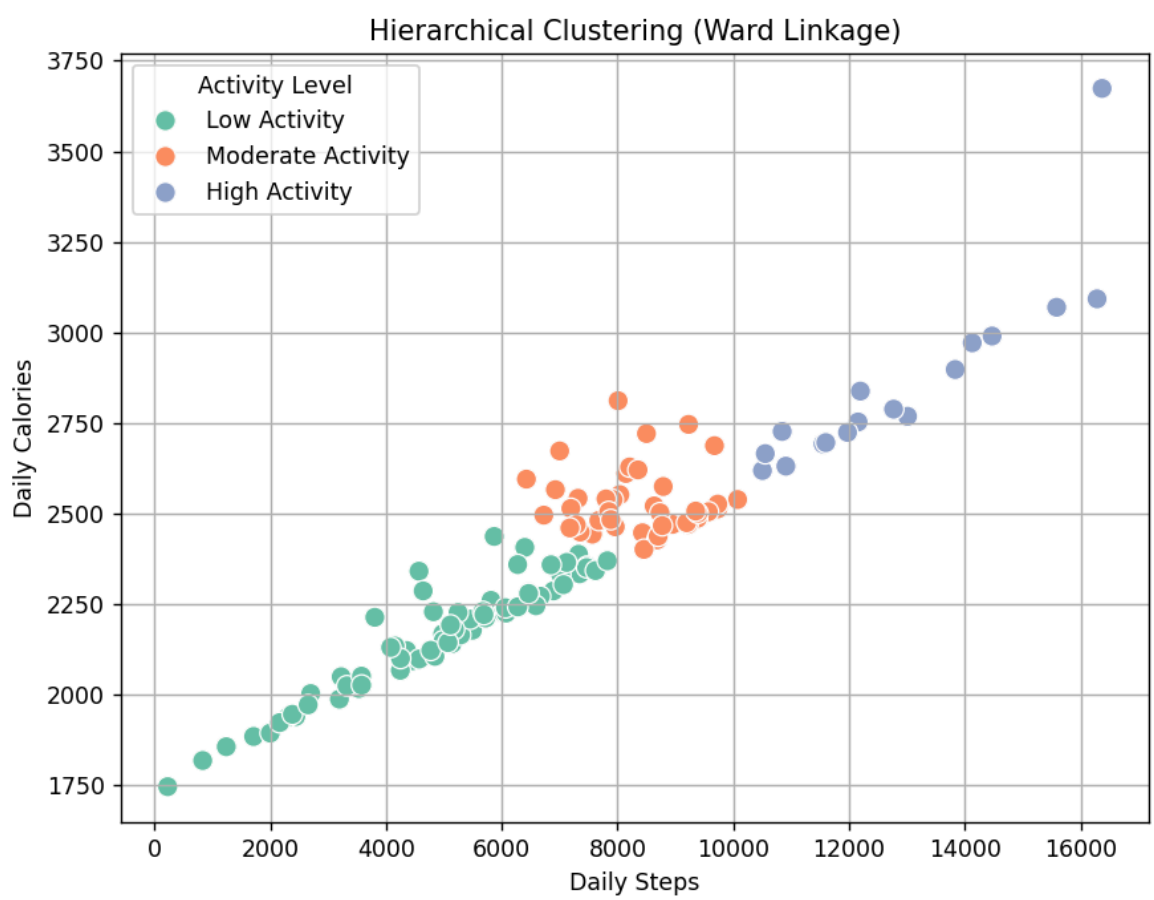

```
Silhouette Score: 0.5409900026472643
```

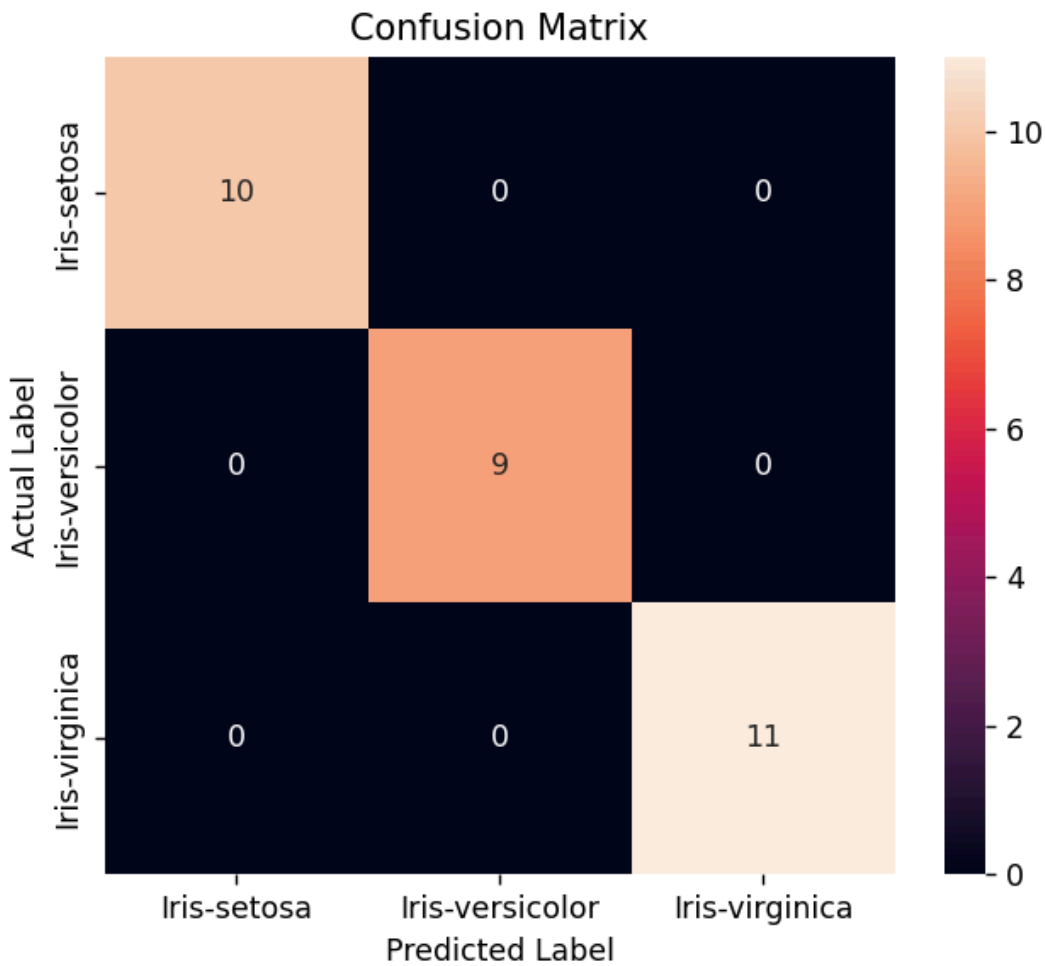
Hierarchical clustering

```python
linked = linkage(X_scaled, method='ward')


plt.figure(figsize=(10,6))
dendrogram(
    linked,
    no_labels=True
)

plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)")
plt.xlabel("Observations")
plt.ylabel("Distance")
plt.grid(True)
plt.show()


daily['Hierarchical_Cluster'] = fcluster(linked, 3, criterion='maxclust')


h_avg = daily.groupby('Hierarchical_Cluster')['steps'].mean()
sorted_h = h_avg.sort_values().index.tolist()

h_labels = {
    sorted_h[0]: "Low Activity",
    sorted_h[1]: "Moderate Activity",
    sorted_h[2]: "High Activity"
}

daily['Hierarchical_Label'] = daily['Hierarchical_Cluster'].map(h_labels)


plt.figure(figsize=(8,6))
sns.scatterplot(
    data=daily,
    x='steps',
    y='calories',
```

```
        hue='Hierarchical_Label',
        palette='Set2',
        s=80
    )

    plt.title("Hierarchical Clustering (Ward Linkage)")
    plt.xlabel("Daily Steps")
    plt.ylabel("Daily Calories")
    plt.legend(title="Activity Level")
    plt.grid(True)
    plt.show()
```

Output:






```
Classification Report:
                 precision    recall  f1-score

    Iris-setosa       1.00      1.00      1.00
Iris-versicolor       1.00      1.00      1.00
 Iris-virginica       1.00      1.00      1.00

       accuracy                           1.00
      macro avg       1.00      1.00      1.00
   weighted avg       1.00      1.00      1.00
```

Conclusion:

Both K-Means and Hierarchical clustering are effective unsupervised learning techniques for grouping similar data points. K-Means is efficient for large datasets and provides clear cluster assignments but requires the number of clusters to be specified beforehand. Hierarchical clustering does not require a predefined number of clusters and produces a dendrogram that gives insights into the data's structure at multiple levels, though it can be computationally intensive for large datasets. The choice between the two depends on dataset size, the need for interpretability, and whether prior knowledge of the number of clusters exists. Proper parameter tuning in both methods enhances cluster quality and meaningful insights.